



A real-time video transcoding and streaming platform built with Go, SQLite, FFmpeg, and React. Transforms any media source into HLS for instant web playback.

1. Table of Contents

Executive Summary

System Architecture

2.1 Architectural Overview

2.2 Data Flow

2.3 Technology Stack

Component Design

3.1 Media Engine

3.2 Job Manager

3.3 Database Layer

3.4 API Layer

3.5 WebSocket Events

Database Schema

4.1 Entity Relationship

4.2 Table Definitions

API Reference

5.1 REST Endpoints

5.2 WebSocket Protocol

5.3 HLS Streaming

UI / UX Design

6.1 Page Structure

6.2 Key Components

Deployment Guide

7.1 Prerequisites

7.2 Installation

7.3 Configuration

Operations Guide

8.1 Health Monitoring

8.2 Resilience Model

8.3 Disk Space Management

2. Executive Summary

vieo is an open-source video streaming platform designed for self-hosted environments. It ingests media from arbitrary sources (files, RTMP streams, RTSP feeds, or capture devices), transcodes them into the HLS (HTTP Live Streaming) format using FFmpeg, and serves the resulting segments over HTTP for playback in any modern web browser via hls.js.

The platform is built around a **channel** abstraction: each source belongs to a channel, and viewers navigate channels to find live or archived content. A robust job-manager oversees the lifecycle of every transcoding operation — from *pending* through *running* to *completed* or *failed* — with automatic pause/resume based on disk pressure, and full recovery after service restarts.

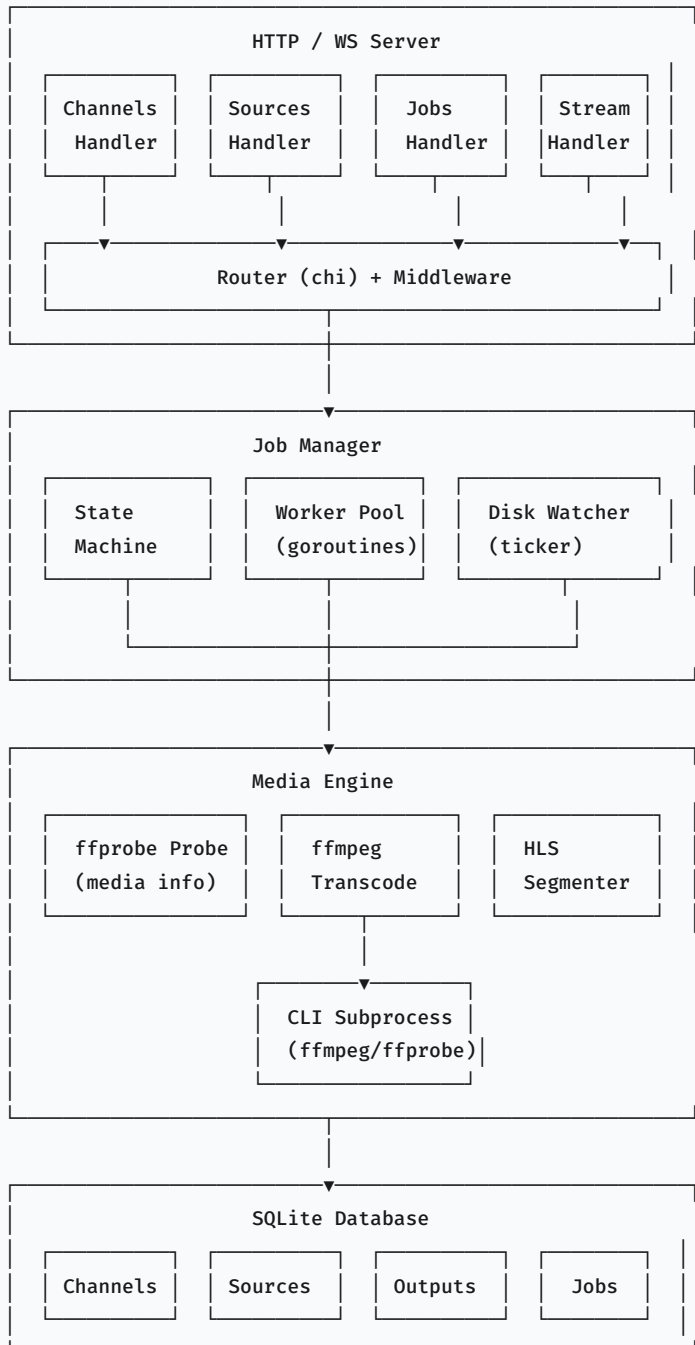
Key design goals include:

- **Single-binary deployment** — Go produces a statically linked binary with no runtime dependencies beyond FFmpeg.
- **Zero external services** — SQLite replaces PostgreSQL/MySQL. No Redis, no message queue.
- **Resilience by default** — Jobs survive restarts, disk pressure is handled gracefully, and the system self-recovers.
- **Beautiful UI** — A shadcn/ui React frontend that feels modern and responsive.

3. System Architecture

3.1 2.1 Architectural Overview

vio follows a **layered monolith** architecture within a single process. All components communicate through Go interfaces, keeping the system testable and maintainable without introducing network overhead between services.



3.2 2.2 Data Flow

1. **Ingest** — A source is registered via the API (file path, RTMP URL, or device).
2. **Probe** — ffprobe extracts codec, resolution, duration, and format metadata.
3. **Transcode** — A job is created and dispatched to the worker pool. ffmpeg reads the source and writes HLS segments.
4. **Serve** — The HTTP server exposes .m3u8 playlists and .ts segments under `/api/stream/{id}/`.

- 5. **Play** — The React frontend uses hls.js to fetch and render the stream in a `<video>` element.
- 6. **Monitor** — WebSocket pushes real-time job progress (status, progress %, logs) to the UI.

Transcoding Pipeline: Source → ffprobe (info) → ffmpeg (decode → encode H.264/AAC) → segmenter (.ts + .m3u8) → HTTP server → hls.js → <video>

3.3 2.3 Technology Stack

Layer	Technology	Rationale
Language	Go 1.22+	Single binary, great concurrency, fast compilation
HTTP Router	chi	Lightweight, stdlib-compatible, middleware ecosystem
Database	SQLite (modernc.org/sqlite)	Zero-dependency, pure Go, embedded
WebSocket	nhooyr.io/websocket	Context-aware, clean API, stdlib-friendly
Media	ffprobe / ffmpeg (CLI)	Battle-tested, supports every format, easy to update
Frontend	React 18 + Vite	Fast dev experience, modern tooling
UI Library	shadcn/ui	Accessible, customizable, beautiful defaults
Player	hls.js	MSE-based HLS playback in all modern browsers

4. Component Design

4.4 3.1 Media Engine

The Media Engine (`internal/media/`) encapsulates all interaction with FFmpeg and ffprobe. It exposes two primary interfaces:

Probe

Wraps `ffprobe` to extract media metadata. Returns a `MediaInfo` struct containing:

- Format: container, duration, bitrate, size
- Video streams: codec, resolution, frame rate, pixel format
- Audio streams: codec, sample rate, channels, bitrate

Transcode

Constructs and executes an `ffmpeg` command that transcodes any input to HLS. The command template:

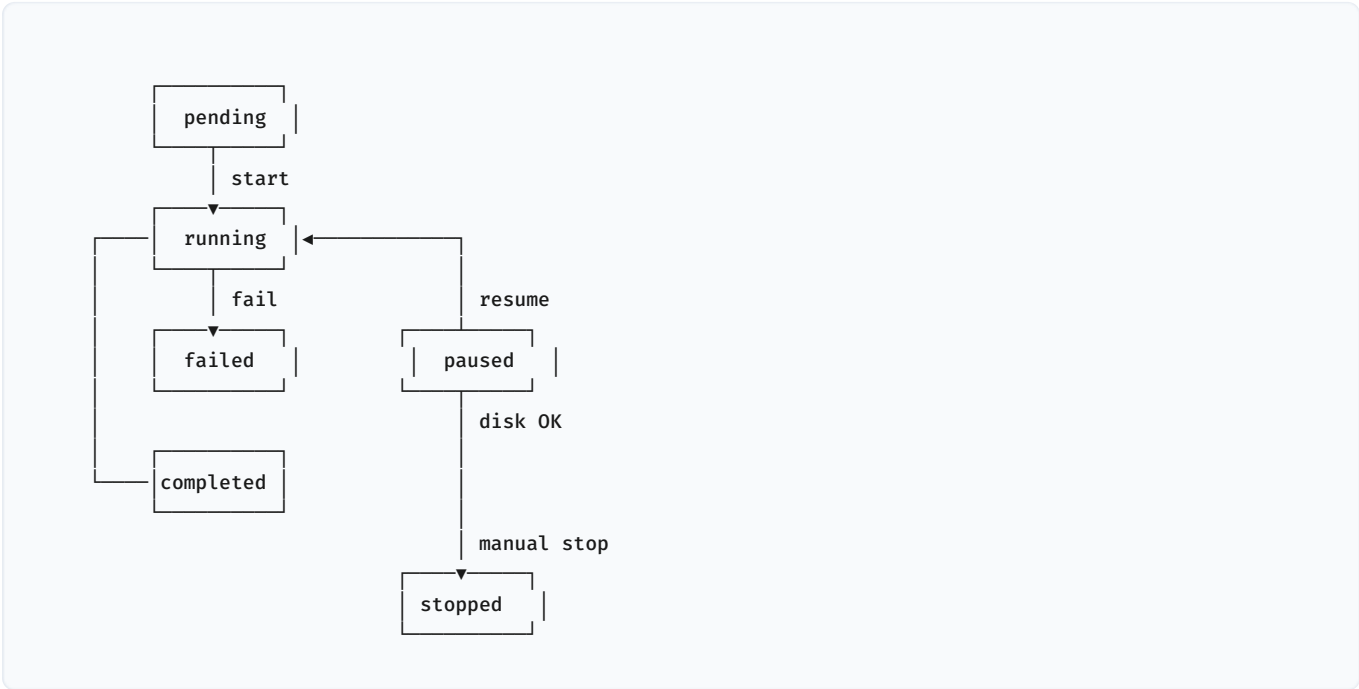
```
ffmpeg -i <input> \  
-codec:v libx264 -preset fast -crf 23 \  
-codec:a aac -b:a 128k \  
-f hls \  
-hls_time 4 \  
-hls_list_size 0 \  
-hls_segment_filename <output_dir>/seg_%05d.ts \  
<output_dir>/playlist.m3u8
```

Progress is parsed from FFmpeg's stderr (the `time=...` lines) and pushed into a channel for the Job Manager to consume.

4.5 3.2 Job Manager

The Job Manager (`internal/job/`) owns the lifecycle of every transcoding operation.

State Machine



State	Description
pending	Job created, awaiting resources
running	Actively transcoding
paused	Paused due to disk pressure or signal

completed	Source fully consumed, HLS output ready
failed	Unrecoverable error (bad source, corrupt file)
stopped	Manually stopped by user

Worker Pool

A configurable number of goroutines (default: 3) consume from a job channel. Each worker manages one ffmpeg subprocess. The pool respects context cancellation for graceful shutdown.

Disk Watcher

A background goroutine ticks every 30 seconds, checking disk usage on the data partition via `syscall.Statfs`. When usage exceeds `VIEO_DISK_WARN` (default 90%), running jobs are paused. Below that threshold, paused jobs resume automatically. At `VIEO_DISK_CRIT` (95%), jobs are force-stopped.

4.6 3.3 Database Layer

Uses `modernc.org/sqlite` — a pure Go SQLite implementation that requires no CGO. Migrations are stored as an ordered slice of SQL strings in `internal/db/db.go` and applied at startup.

Each model (channel, source, output, job) lives in its own file under `internal/db/models/` and exposes CRUD functions that accept `context.Context` and `*sql.DB` as first parameters.

4.7 3.4 API Layer

Built on `go-chi/chi` v5, the API server is defined in `internal/server/server.go` with routes registered in `internal/server/routes.go`. Handlers are grouped by resource in `internal/server/handler/`.

Middleware stack:

- `chi.Logger` — request logging
- `chi.Recoverer` — panic recovery
- `chi.RealIP` — trusted proxy IP
- `CORS` — for frontend dev server (localhost:5173)

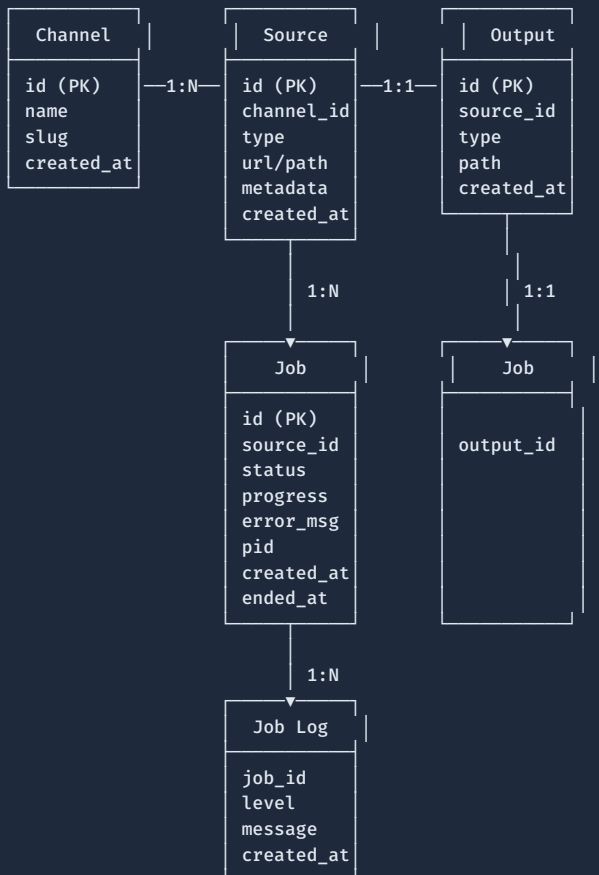
4.8 3.5 WebSocket Events

The `/api/ws` endpoint upgrades HTTP connections to WebSocket. The server pushes typed JSON events to all connected clients:

Event Type	Payload	When
<code>job:update</code>	<code>{"id":1,"status":"running","progress":0.45}</code>	Progress tick (every ~2s)
<code>job:log</code>	<code>{"id":1,"level":"info","message":"..."}</code>	ffmpeg log line
<code>job:complete</code>	<code>{"id":1,"status":"completed"}</code>	Job finishes successfully
<code>job:error</code>	<code>{"id":1,"status":"failed","error":"..."}</code>	Job fails
<code>job:paused</code>	<code>{"id":1,"status":"paused","reason":"disk"}</code>	Job paused

5. Database Schema

5.9 4.1 Entity Relationship



5.10 4.2 Table Definitions

channels

```
CREATE TABLE channels (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT      NOT NULL,
  slug        TEXT      NOT NULL UNIQUE,
  description TEXT      DEFAULT '',
  created_at  TEXT      NOT NULL DEFAULT (datetime('now'))
);
```

sources

```
CREATE TABLE sources (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  channel_id  INTEGER NOT NULL REFERENCES channels(id) ON DELETE CASCADE,
  type        TEXT      NOT NULL CHECK(type IN ('file','rtmp','rtsp','device')),
  url         TEXT      NOT NULL,
  metadata    TEXT      DEFAULT '{}',
  created_at  TEXT      NOT NULL DEFAULT (datetime('now'))
);
```

outputs

```
CREATE TABLE outputs (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  source_id   INTEGER NOT NULL REFERENCES sources(id) ON DELETE CASCADE,
  type        TEXT     NOT NULL DEFAULT 'hls',
  path        TEXT     NOT NULL,
  created_at  TEXT     NOT NULL DEFAULT (datetime('now'))
);
```

jobs

```
CREATE TABLE jobs (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  source_id   INTEGER NOT NULL REFERENCES sources(id) ON DELETE CASCADE,
  output_id   INTEGER NOT NULL REFERENCES outputs(id) ON DELETE CASCADE,
  status      TEXT     NOT NULL DEFAULT 'pending'
                CHECK(status IN ('pending','running','paused','completed','failed','stopped')),
  progress    REAL     NOT NULL DEFAULT 0.0,
  error_msg   TEXT     DEFAULT '',
  pid         INTEGER  DEFAULT 0,
  created_at  TEXT     NOT NULL DEFAULT (datetime('now')),
  ended_at    TEXT
);
```

job_logs

```
CREATE TABLE job_logs (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  job_id       INTEGER NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  level        TEXT     NOT NULL DEFAULT 'info',
  message      TEXT     NOT NULL,
  created_at   TEXT     NOT NULL DEFAULT (datetime('now'))
);
```

6. API Reference

6.11 5.1 REST Endpoints

Health

```
GET /api/health
→ 200 {"status":"ok","version":"1.0.0"}
```

Channels

```
GET    /api/channels      → List all channels
POST   /api/channels      → Create channel
GET    /api/channels/{id} → Get channel by ID
PUT    /api/channels/{id} → Update channel
DELETE /api/channels/{id} → Delete channel (cascades sources, outputs, jobs)
```

Sources

```
GET    /api/sources      → List sources (?channel_id=N to filter)
POST   /api/sources      → Create source
GET    /api/sources/{id} → Get source by ID
DELETE /api/sources/{id} → Delete source (cascades outputs, jobs)
```

Outputs

```
GET    /api/outputs      → List outputs
POST   /api/outputs      → Create output for a source
DELETE /api/outputs/{id} → Delete output
```

Jobs

```
GET    /api/jobs          → List jobs (?status=running&source_id=N)
POST   /api/jobs          → Create + start job
POST   /api/jobs/{id}/stop → Stop job (SIGTERM → SIGKILL)
DELETE /api/jobs/{id}     → Delete job record
```

Streaming

```
GET /api/stream/{id}/*      → Serve HLS files
    /api/stream/{id}/playlist.m3u8
    /api/stream/{id}/seg_00001.ts
```

6.12 5.2 WebSocket Protocol

```
GET /api/ws → Upgrade to WebSocket
```

The WebSocket connection is bidirectional. The server pushes events as JSON objects. All event objects share this structure:

```
{
  "type": "job:update",      // event type
  "payload": {               // event-specific data
    "id": 1,
    "status": "running",
    "progress": 0.45
  }
}
```

Type	Direction	Description
------	-----------	-------------

<code>job:update</code>	Server → Client	Periodic progress update
<code>job:log</code>	Server → Client	Transcoding log line
<code>job:complete</code>	Server → Client	Job completed successfully
<code>job:error</code>	Server → Client	Job failed with error
<code>job:paused</code>	Server → Client	Job paused (disk/network)
<code>ping</code>	Server → Client	Keepalive (every 30s)
<code>pong</code>	Client → Server	Keepalive response

6.13 5.3 HLS Streaming

HLS segments and playlists are served as static files from the data directory. The server maps `/api/stream/{output_id}/*` to the output's path on disk.

Cache headers: Playlists (`.m3u8`) are served with `Cache-Control: no-cache` . Segments (`.ts`) are served with `Cache-Control: public, max-age=3600` .

7. UI / UX Design

7.14 6.1 Page Structure

Route	Page	Description
/	Dashboard	Overview: active jobs, channel list, system health
/channels	Channels	List, create, edit, delete channels
/channels/:id	Channel Detail	Channel info, sources list, video player
/sources	Sources	List, create, delete sources
/jobs	Jobs	List all jobs with status, progress, controls
/jobs/:id	Job Detail	Job logs, progress chart, controls

7.15 6.2 Key Components

VideoPlayer

Wraps hls.js with a clean `<video>` element. Features: play/pause, seek, volume, fullscreen, picture-in-picture. Supports automatic quality switching via hls.js level selection.

JobStatusBadge

A compact badge component showing job state with color coding:

- **Pending** — gray
- **Running** — blue + animated pulse
- **Paused** — amber
- **Completed** — green
- **Failed** — red
- **Stopped** — slate

ProgressBar

A thin animated progress bar (0–100%) that updates via WebSocket in near real-time. Uses `transition: width 0.5s ease` for smooth visual updates.

SidebarLayout

The main application shell with:

- Collapsible sidebar (navigation links, branding)
- Top header bar (breadcrumbs, system status indicator)
- Main content area (router outlet)

8. Deployment Guide

8.16 7.1 Prerequisites

- Go 1.22+
- FFmpeg + ffprobe (available on PATH)
- Node.js 18+ and npm (for frontend builds)
- A POSIX-compatible OS (Linux, macOS, BSD)

8.17 7.2 Installation

```
# 1. Clone repository
git clone https://github.com/arthurkay/vieo
cd vieo

# 2. Build backend
go build -o vieo ./cmd/vieo

# 3. Build frontend
cd web
npm install
npm run build
cd ..

# 4. Run
./vieo \
  --db ./data/vieo.db \
  --data-dir ./data/hls \
  --http-addr :8080
```

The frontend build output is automatically embedded in the Go binary (via `embed.FS`) and served at the root `/` when no other route matches.

8.18 7.3 Configuration

Flag	Env Variable	Default	Description
<code>--db</code>	<code>VIEO_DB_PATH</code>	<code>./vieo.db</code>	SQLite database path
<code>--data-dir</code>	<code>VIEO_DATA_DIR</code>	<code>./data</code>	HLS output directory
<code>--http-addr</code>	<code>VIEO_HTTP_ADDR</code>	<code>:8080</code>	HTTP bind address
<code>--log-level</code>	<code>VIEO_LOG_LEVEL</code>	<code>info</code>	debug, info, warn, error
<code>--disk-warn</code>	<code>VIEO_DISK_WARN</code>	<code>90</code>	Disk % to pause jobs
<code>--disk-crit</code>	<code>VIEO_DISK_CRIT</code>	<code>95</code>	Disk % to stop jobs
<code>--max-jobs</code>	<code>VIEO_MAX_JOBS</code>	<code>3</code>	Max concurrent transcodes

Priority: Flags override env vars, which override defaults. This allows for simple development with defaults and production configuration through the environment.

9. Operations Guide

9.19 8.1 Health Monitoring

The `/api/health` endpoint returns:

```
{
  "status": "ok",
  "version": "1.0.0",
  "uptime": 3600,
  "jobs": {
    "running": 2,
    "pending": 1,
    "completed": 15,
    "failed": 0
  },
  "disk": {
    "usage_percent": 67,
    "total_gb": 500,
    "free_gb": 165
  }
}
```

9.20 8.2 Resilience Model

Job Persistence

At startup, the Job Manager queries the database for all jobs left in `running` or `paused` state and resumes them. Running jobs from a previous process instance are reset to `pending` (since their ffmpeg PID is gone), then restarted.

Graceful Shutdown

On `SIGTERM` / `SIGINT` :

1. HTTP server stops accepting new connections.
2. Running ffmpeg processes receive `SIGTERM` .
3. After 5 seconds, any remaining processes receive `SIGKILL` .
4. Job states are persisted to the database.
5. Database connection is closed cleanly.
6. Process exits.

Process Supervision

If an ffmpeg subprocess exits unexpectedly, the Job Manager marks it as `failed` and logs the exit code. A configurable retry policy (`max_retries: 3` with exponential backoff) can reattempt the job.

9.21 8.3 Disk Space Management

Threshold	Action
< 90%	Normal operation. All jobs proceed.
90% – 95%	Running jobs are paused (SIGSTOP to ffmpeg). Paused jobs resume when disk drops below 85%.
> 95%	Running jobs are stopped (SIGTERM → SIGKILL). No new jobs accepted. Admin alert logged.

The disk watcher checks every 30 seconds using `syscall.Statfs` on the data directory partition.